

• PPS •

Clasificarea lui Flynn:

- SISD (microprocesare clasice cu arh. von Neumann)
- SIMD
- MISD (procesare pipeline)
- MIMD (cluster)

memorie ← partajată
distribuită
hibridă

- shared memory $\Rightarrow$ 2 clase: UMA și NUMA
- cache coherent (dacă un procesor modifică o locație de memorie toate celelalte "stiu" despre această modificare)
- shared memory multiprocessors $\Rightarrow$ SMP
- latență: timpul în care o dată ajunge să fie disponibilă la procesor după ce s-a inițiat cererea
- bandwidth: rata de transfer a datelor din memorie către procesor

Shared Memory:

⊕
Global address space
Partajare date
rapidă și uniformă

⊖
lipsă
scalabilitate
siner. $\rightarrow$ programator
costuri mari

Arhitecturi cu memorie distribuită

⊕
Memorie scalabilă
cost redus

⊖
Comunicațiile - programatorul
Dificil de a mapa struct. de date mari
Acces neuniform la memorie

- massively parallel processor $\Rightarrow$ MPP

- mod are local

{ memorie fizică
spațiu de adresare
disc local și conexiuni la rețea
sistem de operare

Tipuri arhitecturi paralele:

- Uniprocessor
 - scalar processor
 - vector processor
 - SIMD
- SMP - shared memory address space
 - Bus-based memory system
 - interconnection network
- Distributed memory multiprocessors
 - Message passing between nodes
 - MPP
- Cluster of SMPs - shared memory addressing within SMP node
 - message passing between SMP nodes
 - MPP if processor nr. is large and the communication is fast.

Proces: o instanță a unui program care se execută

- ID
- stare
- context
- memorie

Scheduler: controlează execuția proceselor
setează stările procesului

- new
- running
- blocked
- ready
- terminated

Thread: o parte

Race condition: condiție care poate să apară într-un sistem în care comportamentul este dependent de ordinea în care apar anumite evenimente

Data race: un thread exec. o operație prin care se încercă să se acceseze o locație de memorie care este în același timp accesată pt. scriere de alt thread

Secțiune critică: segment de cod unde poate apărea data race

Instrucțiune atomică: dacă execuția sa nu poate fi "interleaved" cu cea a altei instrucțiuni înainte de terminarea ei.

MPI - Structuring program MPI

MPI include file
Declarations, prototypes
Program begins

Serial code

Initialize MPI environment (Parallel code begins)

Do work and make message passing calls

Terminate MPI environment (Parallel code ends)

Serial code

Program ends

```
#include "mpi.h"
```

```
#include <stdio.h>
```

```
int main (int argc, char *argv[])
```

```
{ int numtasks, rank, rc;
```

```
rc = MPI_Init (&argc, &argv);
```

```
if (rc != MPI_SUCCESS) {
```

```
    printf ("Error starting MPI program...");
```

```
    MPI_Abort (MPI_COMM_WORLD, rc);
```

```
}
```

```
MPI_Comm_size (MPI_COMM_WORLD, &numtasks);
```

```
MPI_Comm_rank (MPI_COMM_WORLD, &rank);
```

```
printf ("Nr. tasks = %d My rank = %d\n", numtasks, rank);
```

```
/* do work */
```

```
MPI_Finalize();
```

```
}
```

```
MPI_Send (buffer, count, type, dest, tag, comm)
```

```
MPI_Recv (buffer, count, type, source, tag, comm, status)
```

```
MPI_Barrier (comm, ierr)
```

```
MPI_Bcast (&msg, count MPI_INT, source, MPI_COMM_WORLD)
```

```
MPI_Scatter (sendbuf, sendcnt, MPI_INT, recvbuf, recvcnt, MPI_INT,  
            root, MPI_COMM_WORLD)
```

```
MPI_Gather (==)
```

```
MPI_Reduce (sendbuf, recvbuf, count, MPI_INT, MPI_SUM, dest, comm)
```

Interacțiune între procese/threaduri ← comunicare
sincronizare

Sincronizare {
- **excludere mutuală** (se evită utilizarea simultană de către mai multe procese a unei resurse critice)
- **pe condiție** (se amână execuția unui proces până când o anumită condiție devine TRUE)

Deadlock: un grup de procese/threaduri se blochează la infinit pentru că fiecare proces așteaptă după o resursă care este reținută de alt proces care la rândul lui așteaptă după alta resursă.

Starvation: dacă unui thread nu i se alocă timp de execuție CPU timp pt. că alte threaduri folosesc CPU
Thread este "starved to death" pt. că alte threaduri au acces la CPU în locul lui.
"Fairness" - toate threadurile au șanse egale la CPU

Live-lock: situația în care un grup de procese/threaduri nu progresa datorită faptului că își cedează reciproc execuția.

Operații atomice: sunt efectuate ca o singură unitate - indivizibil
nu parțial, ori complet ori deloc

Semafon: primitivă de sincronizare de nivel înalt (nu cel mai)
2 operații indivizibile: $P(S)$ (apelată de procese care doresc să acceseze rez. critică)

$V(S)$ (apelată la sf. sect. critice și semnifică eliberarea acesteia pt. alte procese)

o variabilă $\rightarrow$ count = $V(S)$ (valoare semafor)

```
class SEMAPHORE feature
  count: INTEGER
  down
    do
      await count > 0
      count := count - 1
    end
  up
    do
      count := count + 1
    end
end
```

Fără o evidență a proceselor care așteaptă intrarea în secțiunea critică!
 $\Rightarrow$ nu se poate asigura starvation-free

Weak Semaphore

$\{V(S), C(S)\}$

- valoarea semaforului
- multime de așteptare la semafor
- + op. $P(S)/\text{down}$ și $V(S)/\text{up}$

Strong Semaphore

$\{V(S), C(S)\}$

- valoarea semaforului
- coada de așteptare, conține ref. la procesele care așteaptă la semaforul S (FIFO)
- + op. P/down și V/up

count: INTEGER

blocked: CONTAINER

down

do

if count > 0 then
count := count - 1

else

blocked.add(P) // procesul curent
P.state := blocked // block process P

end

end

up

do

if blocked.is_empty then
count := count + 1

else

Q := blocked.remove // select some process Q
Q.state := ready // unblock process Q

end

end

weak semaphores $\rightarrow$ posibil starvation (random)

semafor binar $\rightarrow$ mutex

JAVA: java.util.concurrent.Semaphore package

- Constructors: Semaphore(int K)
Semaphore(int K, boolean b) strong = TRUE
- Operations: acquire() (down) throws InterruptedException
release() (up)

dezavantaje

- nu se poate det. utilizarea corectă doar din codul în care apare
- dacă se poziționează încorect o operație P sau V se compromite corectitudinea
- este ușor să se introducă deadlocks în program

Monitor: poate fi considerat un tip abstract de dată (poate fi implementat ca și o clasă) care conține din:

- set permanent de **variabile** = resursa critică
- set de **proceduri** = operații asupra variabilelor
- **corp de inițializare** (secv. de instrucțiuni)
- corpul e apelat la lansarea programului și produce valori initiale pt. variabilele-monitor
- monitorul este accesat numai prin procedurile sale
- numai una din procedurile monitorului poate fi executată la un moment dat $\Rightarrow$ excludere mutuală
- sincronizarea pe condiție $\rightarrow$ variabile de tip condiție
wait signal (notify)
- object-oriented view - toate atributele sunt private
- metodele se exec. prin excludere mutuală

```
monitor class MONITOR_NAME
feature
  -- attribute declarations
  a1: TYPE1
  ...
  -- routine declarations
  r1(arg1, ..., argk) do ... end
  ...
invariant
  -- monitor invariant
end
```

```
entry: SEMAPHORE  n(arg1, ..., argk)
init. v(entry) = 1  do
  entry.down
  body
  entry.up
end
```

Variabile condiționale: o abstracție care permite sincronizarea condițională

- asociate cu lacătul unui monitor
- permit threadurilor să aștepte în interiorul unei secțiuni critice eliberând lacătul monitorului

Variabila condițională: conține într-o coadă de blocare și 3 operații atomice:

- wait (elib. lacătul monitorului, blochează threadul care se execută și-l adaugă în coadă)
- signal (dacă coada este empty are efect altfel deblochează un thread din coadă)
- is_empty (TRUE/FALSE, coada e empty?)
- wait + signal $\rightarrow$ din corpul unei rutine a mon.

class CONDITION_VARIABLE

feature

blocked: QUEUE

wait

do

entry.up // release the lock on the monitor

blocked.add(P) // P - the current process

P.state := blocked // block P

end

signal deferred end // behavior depends on signaling discipline

is_empty: BOOLEAN

do

result := blocked.is_empty

end

end

JAVA: fiecare obiect are un monitor care poate fi blocat sau deblocat în blocurile sincronizate
variabilele conditionale nu sunt explicit disponibile, dar metodele { wait(), notify(), notifyAll() } pot fi apelate din orice bloc sincronizat

nu sunt mai multe var. cond. asociate unui monitor
disciplina = 'Signal and Continue'
monitoarele nu sunt starvation-free

avantaje: abordare structurată
separation of concerns

probleme: trade-off → suport pt. programator în performanță
disciplinele de semnalizare - surse de confuzie
nested monitor calls

Forme de sincronizare JAVA: Synchronized Static Methods
Nonblocking Counter - Atomic variables
Lock
Semaphore
Exchanger
Class Synchronizer
ReadWriteLock
CyclicBarrier

Call by future:

- non-deterministic: valoarea se va calcula cândva între momentul creării variabilei future și momentul când aceasta se va folosi
- eager evaluation: imediat ce future a fost creată
- lazy evaluation: doar atunci când e folosită

Future and Promise:

- the two sides of an asynchronous operation
- a caller of an asynchronous task will get a Future as a handle to the computation's result

C++ 11

- future {
 - promise
 - async
 - packaged_task}

- `std::packaged_task` object = wraps a callable object
- `async` = execută o funcție f asincron posibil în alt thread și returnează un obiect `std::future` care va conține rezultatul

• depinde de implementare dacă `std::async` pornește un nou thread sau dacă task-ul se va executa sincron atunci când se va cere valoarea pt. future.

- `std::promise` = furnizează un mecanism de a stoca o valoare sau o excepție care va fi apoi obținută asincron via un obiect `std::future` care a fost creat prin obiectul promise

• activări: {

- make ready
- release
- abandon

}

JAVA

- {
 - task (Runnable vs. Callable)
 - Future
 - Executor
 - CompletableFuture}

- `task` = activitate independentă, nu depinde de starea, rezultatul ori 'side effects' ale altor task-uri
- `Executor` = mecanism de decuplare a submiterii unui task de execuția lui, suport pt. monitorizarea execuției, se bazează pe 'modelul producător-consumator'

OpenMP - structural general code

```
#include <omp.h>
```

```
main() {
```

```
    int var1, var2, var3;
```

```
    Serial Code
```

```
    /* Beginning of parallel section. Fork a team of threads.
       Specify variable scoping */
```

```
    #pragma omp parallel private(var1, var2) shared(var3)
```

```
    {
        /* Parallel section executed by all threads */
```

```
        ...
        /* All threads join master thread and disband */
```

```
    }
    Resume serial code
    ...
}
```

Example - PARALLEL Region

```
#include <omp.h>
```

```
main() {
```

```
    int nthreads, tid;
```

```
    omp_set_num_threads(n);
```

```
    #pragma omp parallel private(tid)
```

```
    {
        tid = omp_get_thread_num();
```

```
        printf("Hello from %d\n", tid);
```

```
        if (tid == 0) {
```

```
            nthreads = omp_get_num_threads();
```

```
            printf("Nr. threads = %d\n", nthreads);
```

```
        }
```

```
    } /* All threads join master thread
       and terminate */
```

```
}
```

reduction (+: sum)

```
#pragma omp for [clause...]
```

```
#pragma omp section [clause...]
```

```
#pragma omp single [clause...]
```

```
schedule (static | dynamic |
          guided [chunk])
```

Matrix Multiply:

```
#pragma omp parallel for
private(j, k)
```

```
for(i=0; i<m; i++)
```

```
for(j=0; j<m; j++)
```

```
    c[i][j] = 0;
```

```
for(k=0; k<m; k++)
```

```
    c[i][j] = c[i][j] + a[i][k] * b[k][j]
```

Resumet: #pragma omp parallel - require parallel
- clause

#pragma omp for - clause
- reduce

#pragma omp sections - clause

#pragma omp barrier

#pragma omp critical

Time de execuție: $T_p = (\max_{i: i \in \overline{0, p-1}} T_{\text{calcul}}^i + T_{\text{comunicare}}^i + T_{\text{testare}}^i)$

- complexitatea timp pt. un algoritm paralel care rezolvă o probl. $P(m)$ cu dim. m a datelor de intrare este o funcție T ce depinde de m , dor. nr. de nr. de procesoare p folosite.

pași calcul + pași comunicare / acces la memorie

Overhead:

- $T_{\text{all}} = \text{timp total}$
- $T_s = \text{timp serial}$
- $T_{\text{all}} = p T_p$ ($p = \text{nr. procesoare}$)
- $T_o = p T_p - T_s$

Accelerarea ("speed-up"): raportul dintre timpul de execuție al celui mai bun algoritmul serial cunoscut, executat pe un calculator monoprocesor și timpul de execuție al programului paralel echivalent, executat pe un sistem de calcul paralel,

- $S_p(m) = \frac{T_s(m)}{T_p(m)}$ - m dim. datelor de intrare
- p nr. procesoare folosite

Eficiența: parametru care măsoară gradul de folosire a procesoarelor

- $E = S_p / p$

Legea lui Amdahl: accelerarea procesării depinde de raportul părții secerentiale față de cea paralelizabilă

- seq (e.g. 20%) $\Rightarrow \text{seq} = 20/100$

- par (e.g. 80%) $\Rightarrow \text{par} = 80/100$

calculul serial $T_s = \text{seq} + \text{par} = 1$ unitate

$$\boxed{\text{Speedup} = 1 / (\text{seq} + \text{par} / p)}$$

$\text{par} = (1 - \text{seq})$, $p \# \text{ procesoare}$

$p \rightarrow \infty \Rightarrow S \sim 1 / \text{seq}$ (e.g. $S \sim 100/20 = 5$)

limita superioară a accelerării este dată de fracția părții secerentiale

Legea lui Gustafson: atunci când dimensiunea problemei crește, partea serială se micșorează în procent

- m - dim. probl., $p \# \text{ procesoare}$

$\text{seq}(m) = \text{fracția calculului serial}$

$\text{par}(m) = \text{fracția calculului paralel}$

$T_p = \text{seq}(m) + \text{par}(m) = 1$ $T_s = \text{seq}(m) + p * \text{par}(m)$

$$\boxed{\text{Speedup} = T_s / T_p = \text{seq}(m) + p * \text{par}(m)} \quad \left| \begin{array}{l} \text{seq}(m) \rightarrow 0 \\ \Rightarrow \text{acc. liniară} \end{array} \right.$$

Time de comunicare: $t_{comm} = t_s + t_h \cdot l + t_w \cdot m$

$$\begin{cases} t_s = \text{startup time} \\ t_h = \text{per-hop time} \\ t_w = \text{per-word transfer time} \end{cases}$$

Costul: produsul dintre timpul de execuție și numărul maxim de procesare care se folosește.

$$C_p(m) = t_p(m) \cdot p$$

- o aplicație paralelă este **optimă** d.p.d.v. al costului dacă valoarea acestuia este egală, sau este de același ordin de mărime cu timpul celei mai bune variante secerentiale $C_p = O(t_s)$.

- aplicația este **eficientă** d.p.d.v. al costului $\rightarrow C_p = O(t_s \log p)$

- costul unui sistem paralel ($p \times T_p$) $\rightarrow$ reflectă suma timpului pe care fiecare procesor îl petrece în rezolvarea problemei.

- un sistem paralel se numește optimal dacă costul rez. unei probl. pe un calculator paralel este asimptotic egal cu costul serial $E = T_s / p T_p \Rightarrow$ optimal $E = O(1)$.

Scalabilitatea: este un parametru calitativ care caracterizează atât sistemele paralele, cât și aplicațiile paralele.

- scalabilitatea **aplicației**: abilitatea unui program paralel să obțină o creștere de performanță proporțională cu nr. de procesare și dim. problemei

- scalabilitatea măsoară modul în care se schimbă (crește) performanța unui anumit algoritm în cazul în care sunt folosite mai multe elem. de procesare.

- scalabilitatea unui **sistem paralel**: (softwară + arhitectură) este o măsură a capacității de a lăsa o accelerare cu o creștere liniară în funcție de nr. de procesare folosite.

- metrice pentru scalabilitate:

$$\begin{cases} - \text{funcția de ineficiență} \\ - \text{eficiența} \\ - \text{fracția serială} \end{cases}$$

Granularitatea: este un parametru calitativ care caracterizează atât sistemele paralele, cât și aplicațiile paralele.

- granularitatea aplicației: dimensiunea minimă a unei unități secvențiale dintr-un program, exprimată în nr. de instrucțiuni
- unitate secvențială: parte din program în care nu au loc operații de sincronizare sau comunicare cu alte procese.
- granularitatea unui algoritmul poate fi aproximată ca fiind raportul dintre timpul total de calcul și timpul total de comunicare
- granularitatea sistemului: pt. un sistem paralel, există o valoare minimă a granularității aplicației sub care performanța scade semnificativ. Această valoare de prag este cunoscută ca și granularitatea sistemului respectiv.
- de dorit: un calculator paralel să aibă granular. mică, a.î. să poată executa eficient o gamă largă de programe $\Rightarrow$ programele paralele să fie caracterizate de o granular. mare, a.î. să poată fi executate eficient de o diversitate de sisteme
- nr. de task-uri în care o problemă se descompune determină granularitatea:

$\begin{cases} \text{nr. mare} \Rightarrow \text{fine-grained decomposition} \\ \text{nr. mic} \Rightarrow \text{coarse-grained decomposition} \end{cases}$

- granularitatea este det. de nr. de task-uri care se creează pt. o problemă (multe $\Rightarrow$ granular. mică)
- de multe ori, folosirea a mai puține procesoare îmbunătățește performanța sistemului paralel

Partitionarea: are în vedere împărțirea problemei de programare în componente care se pot executa concurrent.

- două strategii principale de partitionare:
 - $\begin{cases} \text{descompunerea domeniului de date} \\ \text{descompunerea funcțională} \end{cases}$

Decompunerea domeniului de date : aplicabilă atunci când domeniul datelor e mare și regulat, ideea centrală este de a divide domeniul de date, reprezentat de principalele structuri de date, în componente care pot fi manipulate independent

• distribuții de date :

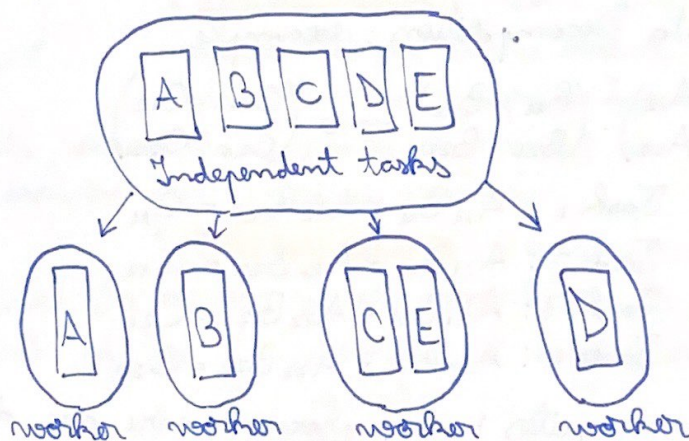
tehnica tăierii (distribuție liniară)

tehnica încrucișării (distribuție ricică)

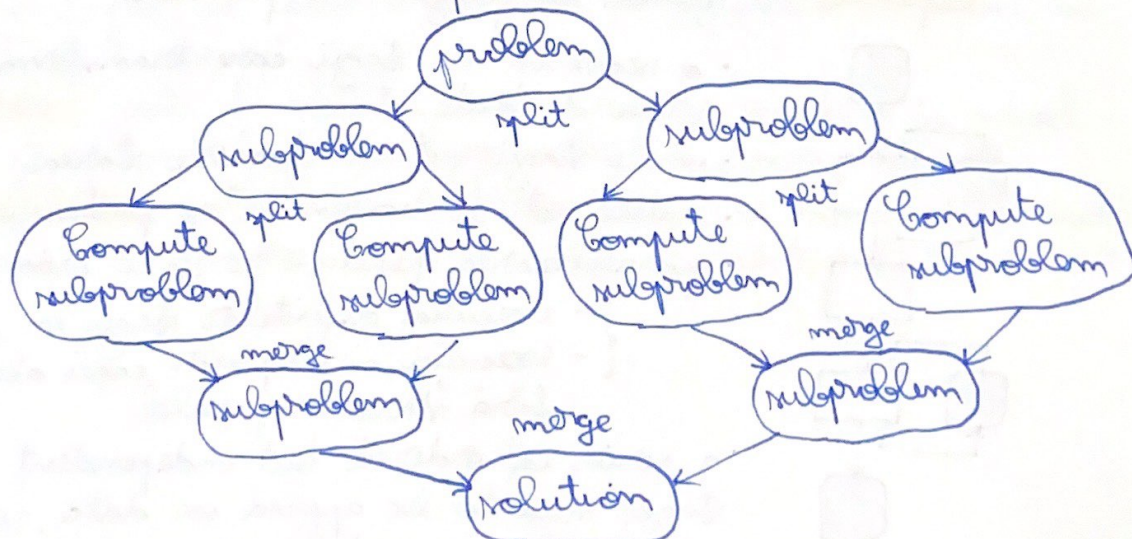
Decompunerea funcțională : tehnică de partitionare folosită atunci când aspectul dominant al problemei este funcția, algoritmul, mai degrabă decât operațiile asupra datelor.

Tablouri de programare paralelă

Master-slave

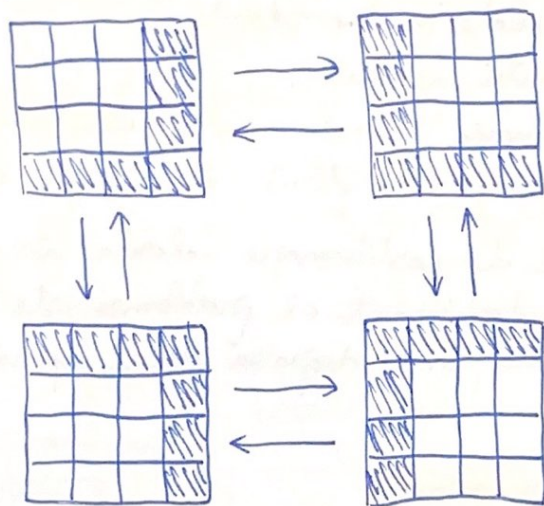


Divide et Impera



Data decomposition: decompunere geometrică

- există dependențe dar comunicarea se face într-un mod predictibil (geometric) → vecini



Output data Decomposition: example

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \rightarrow \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$\text{Task 1: } A_{11}B_{11} + A_{12}B_{21} = C_{11}$$

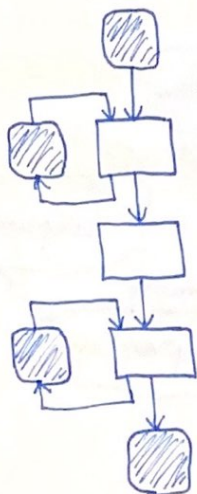
$$\text{Task 2: } A_{11}B_{12} + A_{12}B_{22} = C_{12}$$

$$\text{Task 3: } A_{21}B_{11} + A_{22}B_{21} = C_{21}$$

$$\text{Task 4: } A_{21}B_{12} + A_{22}B_{22} = C_{22}$$

- the owner computes rule: procesul care are data asignată lui este responsabil de calculele asociate acelei date

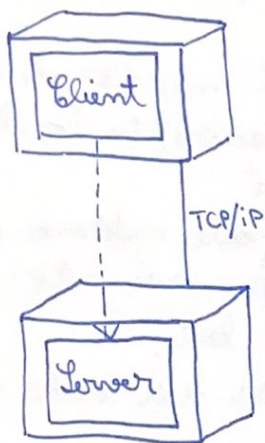
Pipeline → tablou de programare paralelă



- o secvență de stagii care transformă un flux de date
- unele stagii pot să stocheze starea
- datele pot fi "consumate" și produse incremental
- paralelizarea pipeline se face prin:
 - execuția diferitelor stagii în paralel
 - execuția multiplelor copii ale stagiilor fără stare în paralel
- a series of ordered but independent computation stages need to be applied on data, where each output of a computation becomes input of subsequent computation.

Sistem distribuit: este format din componente hardware si software localizate într-o rețea de calculatoare care comunică si își coordonează acțiunile doar bazat pe transmitere de mesaje.

Client-Server pattern



- o componentă de tip server care furnizează servicii către mai multe componente client
- o componentă client care solicită servicii de la componenta server
- serverele sunt active permanent 'ascultând' cererile de la clienți
- clienții și serverele lucrează în sesiuni
 - * **stateless server** (starea unei sesiuni e gestionată de către client. Aceasta stare este trimisă împreună cu fiecare cerere.

În aplicațiile WEB, session state poate fi stocată ca și parametrii URL, în câmpuri ascunse sau folosind cookies

* **stateful server** (starea unei sesiuni este menținută la nivel de server și este asociată cu ID-ul clientului)

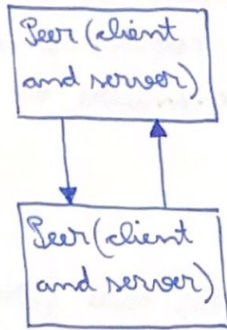
- tranzacțiile treb. să fie
 - atomice, să asigure consistența stării
 - izolate
 - durabile

• **fault-handling** = starea menținută la nivelul clientului implică faptul că toată informația se va pierde în cazul în care clientul eșuează.

• **securitatea** poate fi afectată dacă starea se menține la nivel client pt. că informația se transmite la fiecare request.

• **scalabilitatea** poate fi redusă dacă starea se menține la nivelul serverului 'in-memory'; mulți clienți => necesar memorie crescută.

Peer to peer pattern



• model client-server simetric:

- un **nod** (peer) poate funcționa ca și un client - cere servicii de la alte componente sau ca și server - furnizează servicii pentru alții.
- nodul unui nod se poate schimba în mod dinamic.

- atât clientii, cât și serverele folosesc usual multithreading
- serviciile pot fi implicite (e.g. stream de conectare) în locul unei cereri (request) trimise prin însoțire directă
- un peer care acționează ca și server își poate informa colegii care acționează ca și clienți de apariția unor evenimente; clienții pot fi informați folosind de ex. un event-bus.
- performanța crește odată cu nr. de noduri dar scade atunci când sunt prea puține

• **avantaje**

- nodurile pot folosi capacitatea întregului sistem (cost individual mic cu beneficiu mare obținut prin partajare)
- overheadul de administrare este scăzut (rețelele peer-to-peer se organizează intern)
- scalabilitate foarte bună, reziliență la failure-ul comp. individuale
- configurarea sistemului se poate schimba dinamic

• **dezavantaje**

- nu există garanția calității serviciilor
- nu există garanția securității